Design and Apply Context-Aware Access Controls Based on User Identity, Device Posture, Location, and Application Risk

Team Leader: Vikashini S

Team Member: Brindha T, Janani S.B

Department: B.Sc Computer Science with Data Analytics

College: St. Joseph's College for Women

Abstract

Zero Trust Architecture (ZTA) is a modern cybersecurity model that eliminates implicit trust in network environments.
This project implements a Context-Aware Zero Trust Access Control System using Python, Flask, and SQLite.
The system evaluates contextual attributes such as device security, location, user role, and application risk
to dynamically determine whether access should be granted, restricted, or denied.
The project demonstrates how risk-based authentication and adaptive policy enforcement can strengthen security.

With the rapid growth of digital systems and cloud-based services, traditional perimeter-based security models are no longer sufficient to protect modern networks. The **Zero Trust security model** has emerged as an effective approach that assumes no user or device should be trusted by default, regardless of whether they are inside or outside the network. Instead, every access request must be continuously verified based on multiple contextual factors.

This project presents a **Context-Aware Zero Trust Access Control System** developed using **Python, Flask, and SQLite**. The system evaluates user access requests by considering contextual attributes such as **user identity, device security status, network location, application risk level, and user role**. A **Risk Engine** calculates a dynamic risk score based on these parameters, while a **Policy Engine** enforces predefined security policies to determine the appropriate access decision.

Depending on the calculated risk score, the system may grant **full access, restricted access, require additional verification, or deny access completely**. This approach demonstrates how adaptive and context-aware access control mechanisms can enhance security compared to traditional authentication systems.

1. Introduction

Traditional network security relied heavily on perimeter-based protection, assuming that users within the network could be trusted. However, modern cyber threats, remote work environments, and cloud infrastructure have rendered this approach insufficient. The Zero Trust model follows the principle 'Never Trust, Always Verify', meaning that every user and device must be continuously validated before access is granted.

With the increasing dependence on digital systems, cloud services, and remote work environments, traditional network security models are becoming less effective. Conventional security approaches typically rely on a **perimeter-based model**, where users inside the network are trusted by default while external users are considered untrusted. However, modern cyber threats such as insider attacks, credential theft, and compromised devices have shown that this approach is no longer sufficient to protect sensitive data and resources.

To address these challenges, the **Zero Trust security model** has emerged as a modern cybersecurity framework. The Zero Trust principle follows the concept of **"Never Trust, Always Verify."** This means that every user, device, and system attempting to access resources must be verified and validated before access is granted, regardless of whether the request originates from inside or outside the network.

A key feature of Zero Trust systems is the use of **context-aware authentication**. Instead of relying solely on usernames and passwords, modern access control systems evaluate additional contextual information such as **device security status, user role, geographic location, network type, and application risk level**. By analyzing these factors, the system can dynamically determine the risk associated with each access request.

This project focuses on developing a **Context-Aware Zero Trust Access Control System** using **Python and Flask**. The system simulates a real-world Zero Trust environment where user authentication is combined with contextual risk evaluation. A **Risk Engine** calculates a risk score based on multiple contextual parameters, and a

Policy Engine applies predefined security rules to determine whether access should be granted, restricted, or denied.

The implementation demonstrates how modern access control systems can improve security by continuously verifying users and adapting access decisions based on risk levels.

Traditional network security models relied heavily on **perimeter-based protection**, where systems inside the network were automatically trusted while external entities were considered untrusted. However, with the rapid advancement of digital technologies, cloud computing, and remote work environments, this approach has become increasingly ineffective. Modern organizations operate in highly distributed environments where users access systems from multiple locations and devices. As a result, traditional security models are no longer sufficient to protect sensitive data and resources.

The **Zero Trust security model** has emerged as a modern approach to address these challenges. Zero Trust operates on the principle of **"Never Trust, Always Verify."** This means that no user or device should be automatically trusted, regardless of whether they are inside or outside the network. Every access request must be verified and validated before permission is granted.

One of the key aspects of Zero Trust security is **context-aware authentication**. Instead of relying solely on usernames and passwords, modern access control systems evaluate additional contextual information such as device security status, user role, geographic location, network type, and application risk level. By analyzing these contextual factors, the system can determine the level of risk associated with each access request.

This project focuses on the development of a **Context-Aware Zero Trust Access Control System** using **Python and the Flask web framework**. The system simulates a real-world Zero Trust environment where user authentication is combined with contextual risk evaluation. A **Risk Engine** calculates a risk score based on contextual parameters, and a **Policy Engine** applies predefined security rules to determine whether access should be granted, restricted, or denied.

The implementation demonstrates how modern access control systems can improve cybersecurity by continuously verifying users and dynamically adapting access decisions based on contextual risk levels.

2. Problem Statement

Traditional authentication systems mainly rely on **username and password-based login mechanisms**. While this method is simple and widely used, it is highly vulnerable to security threats such as phishing attacks, password leaks, brute-force attacks, and insider threats. Once a user successfully logs in, they are often granted unrestricted access to network resources without further verification.

Another major issue with traditional systems is the **lack of contextual awareness**. These systems do not consider important factors such as the device being used, the user's geographic location, the security state of the device, or the risk level of the application being accessed. As a result, malicious actors may gain access using stolen credentials without being detected.

In addition, modern organizations face increasing challenges due to **remote work environments, cloud computing, and mobile devices**, which extend the network beyond traditional boundaries. This makes it difficult to rely solely on perimeter-based security solutions.

Therefore, there is a need for an **adaptive access control system** that can evaluate multiple contextual factors before granting access. Such a system should be able to identify potentially risky situations and apply appropriate security measures such as restricting access or blocking unauthorized requests.

This project aims to address these problems by implementing a **Context-Aware Zero Trust Access Control System** that evaluates contextual risk and dynamically enforces access control policies.

Traditional authentication systems primarily rely on **username and password-based login mechanisms**. Although this method is widely used, it is highly vulnerable to various security threats such as phishing attacks, password leaks, brute-force attacks, and insider threats. Once a user successfully logs in, they are often granted full access to system resources without further verification.

Another significant limitation of traditional systems is the **lack of contextual awareness**. These systems typically do not consider important contextual factors such as the device being used, the geographic location of the user, the security status of the device, or the risk level of the application being accessed. As a result, malicious actors may gain access to sensitive systems using stolen credentials without being detected.

In addition, the rise of **remote work environments, cloud services, and mobile devices** has extended organizational networks beyond traditional boundaries. This makes it increasingly difficult to rely solely on perimeter-based security solutions.

Therefore, there is a need for an **adaptive access control system** that evaluates contextual factors before granting access. Such a system should be capable of identifying potentially risky situations and applying appropriate security measures such as restricting access or blocking unauthorized users.

This project addresses these challenges by implementing a **Context-Aware Zero Trust Access Control System** that evaluates contextual risk and dynamically enforces access control policies.

3. Objectives

The main objectives of this project are as follows:

To design and implement a **Zero Trust Access Control System** that follows modern cybersecurity principles.

To develop a **context-aware authentication mechanism** that considers multiple factors such as device status, location, application risk, and user role.

To implement a **Risk Engine** that calculates a dynamic risk score based on contextual parameters.

To create a **Policy Engine** that determines access decisions based on predefined security policies.

To simulate different access scenarios and demonstrate how contextual risk affects access decisions.

To develop a **web-based prototype using Python Flask** that provides a user-friendly interface for testing the system.

To demonstrate how Zero Trust principles can improve security in modern computing environments.

To design and implement a **Zero Trust Access Control System** based on modern cybersecurity principles.

To develop a **context-aware authentication mechanism** that considers multiple contextual factors such as device status, geographic location, application risk level, and user role.

To implement a **Risk Engine** that calculates a dynamic risk score based on contextual parameters.

To develop a **Policy Engine** that determines access decisions based on predefined security policies.

To simulate different access scenarios in order to demonstrate how contextual risk influences access decisions.

To create a **web-based prototype application using Python Flask** that allows users to interact with the system.

To demonstrate the effectiveness of **Zero Trust security principles** in improving access control mechanisms.

4. Scope of the Project

The scope of this project focuses on developing a **prototype implementation** of a Zero Trust access control system for educational and research purposes. The system demonstrates how contextual information can be used to evaluate risk and enforce adaptive access policies.

The project includes the following components:

A **web-based login interface** where users provide their credentials and contextual information.

A **Risk Engine** that calculates a risk score based on factors such as device status, location, application risk, and user role.

A **Policy Engine** that determines whether the user should receive full access, restricted access, or be denied access.

A **database system** for storing user information and roles.

The system is implemented using **Python Flask and SQLite**, making it lightweight and easy to deploy for demonstration purposes.

However, the project does not include advanced enterprise-level security features such as biometric authentication, real-time threat intelligence, or machine learning-based risk analysis. These features can be considered for future improvements.

Overall, the project aims to provide a **conceptual and practical demonstration of Zero Trust security principles**.

The **scope of the project** defines the **boundaries, objectives, features, and limitations** of a project. It explains **what the project will include and what it will not include**. In simple terms, the project scope describes **the work that must be done to successfully complete the project and deliver the expected results**.

The scope helps project teams, stakeholders, and clients clearly understand the **goals, deliverables, tasks, resources, and timeline** of the project. A well-defined project scope prevents misunderstandings, controls project changes, and ensures that the project stays on track.

Project Objectives

The scope describes the **main goals or objectives** that the project aims to achieve. These objectives define **why the project is being carried out** and what problem it intends to solve.

For example, if the project is a software system, the objective might be to **develop a system that automates tasks, improves efficiency, and reduces manual work**.

Clear objectives help guide the project team and ensure that all activities contribute toward achieving the final goal.

Project Deliverables

Deliverables are the **outputs or results that the project will produce**. These can be products, services, documents, or systems that are completed during the project.

Examples of deliverables include:

A software application

A website

System documentation

User manuals

Final project report

The scope clearly states **what will be delivered at the end of the project**.

Features and Functionalities (In-Scope)

This part explains **what tasks, features, or services will be included in the project**.

For example, in a system development project, the scope may include:

User registration and login

Data storage and management

Report generation

Admin control panel

These are the **functions the system will perform**.

Limitations (Out-of-Scope)

The scope also identifies **what is not included in the project**. This prevents confusion and avoids unnecessary expansion of the project work.

For example:

The system may work only on **web browsers**, not mobile apps.

The project may support **a limited number of users**.

Some advanced features may not be included due to time or resource constraints.

These limitations define the **boundaries of the project**.

Stakeholders

The project scope may also mention the **people or groups involved in the project**, such as:

Project manager

Development team

Clients

End users

Organization management

Stakeholders are important because they **influence project decisions and requirements**.

Time and Resource Constraints

The scope often includes **time frame and resource limitations**. This defines:

Project start and end dates

Budget limitations

Available technology or tools

Team size

These constraints help ensure the project is **completed within the available resources**.

Library Management System – Scope

The scope of the Library Management System project is to develop a computerized system that manages library operations efficiently. The system will help librarians manage books, members, and borrowing activities in an organized manner.

The system will allow the librarian to add, update, and delete book records. It will also enable the management of student or member information. Users will be able to search for books based on title, author, or category.

The system will track book borrowing and returning activities and maintain records of issued books. It will also generate reports related to book availability, issued books, and member information.

The project focuses on improving library efficiency by reducing manual record keeping and minimizing errors. However, advanced features such as online book reservations, mobile applications, and multi-branch library management are not included in the project scope.

Online Shopping System – Scope

The scope of the Online Shopping System project is to develop a web-based platform that allows users to browse products and purchase items online.

The system will allow users to create accounts, log in, view products, and add items to a shopping cart. Customers will be able to place orders and view their purchase history.

Administrators will have the ability to manage product information, update prices, and monitor customer orders. The system will maintain a database that stores customer details, product information, and order records.

The project focuses on providing basic e-commerce functionality. Features such as advanced payment gateways, international shipping systems, and AI-based recommendations are outside the scope of this project.

Student Management System – Scope

The scope of the Student Management System project is to develop a system that helps educational institutions manage student information efficiently.

The system will allow administrators to add, update, and manage student records. It will store important details such as student personal information, academic records, and course details.

Teachers and administrators will be able to view student data, update academic performance, and generate reports related to student progress.

The system aims to improve the organization and management of student data while reducing manual paperwork. However, features such as online learning platforms, video classes, and external educational integrations are not included in the scope of this project.

5. Literature Review

The concept of **Zero Trust security** was first introduced by John Kindervag in 2010 while working at Forrester Research. The Zero Trust model challenges the traditional assumption that users within an organization's network can be automatically trusted.

Instead, it emphasizes strict identity verification and continuous monitoring of all access requests.

Several research studies have explored the benefits of Zero Trust architectures in improving cybersecurity. According to **NIST Special Publication 800-207**, Zero Trust Architecture is designed to protect modern enterprise environments that involve cloud services, remote users, and mobile devices. The framework emphasizes identity-based access control, least-privilege access, and continuous authentication.

Recent studies have also highlighted the importance of **context-aware access control systems**. These systems use contextual information such as location, device type, time of access, and user behavior to determine the level of trust associated with an access request.

Risk-based authentication systems have also gained popularity in modern cybersecurity frameworks. These systems assign a **risk score** to each access attempt and enforce security policies based on the level of risk. High-risk scenarios may require additional verification steps such as multi-factor authentication.

Various technologies such as **identity management systems, behavioral analytics, and artificial intelligence** are being integrated with Zero Trust architectures to enhance security. These advancements enable organizations to detect anomalies and prevent unauthorized access more effectively.

This project builds upon these concepts by implementing a simplified Zero Trust system that demonstrates **risk-based authentication and context-aware access control** in a web-based environment.

A **Literature Review** is a section of a research paper or project report where the researcher studies and analyzes **existing research, books, articles, journals, and previous studies related to the project topic**.

The main purpose of a literature review is to **understand what work has already been done in the field** and to identify **gaps or areas that need improvement**. It helps the

researcher build knowledge about the topic and develop a strong foundation for the current project.

In simple terms, a literature review means **reviewing and summarizing the work of other researchers related to the same topic**.

Purpose of Literature Review

The literature review serves several important purposes in research and project development.

Understanding Previous Research

It helps the researcher learn about **previous studies, systems, and methods** developed by other researchers.

Identifying Research Gaps

By studying previous work, researchers can find **limitations or missing features** that their project can improve.

Building Theoretical Knowledge

Literature reviews provide **background knowledge and theoretical understanding** about the topic.

Avoiding Duplication

It ensures that the researcher **does not repeat work that has already been done**.

Supporting the Project Idea

It helps justify **why the current project is necessary and important**.

Sources of Literature Review

Researchers collect literature from different **reliable academic and professional sources**, such as:

- **Books** – Provide fundamental knowledge and theories.
- **Research Papers** – Published studies in journals and conferences.
- **Online Databases** – Google Scholar, IEEE, ScienceDirect, etc.
- **Project Reports and Theses** – Previous academic work.
- **Articles and Technical Blogs** – Provide updated information and trends.

These sources help researchers **analyze different approaches and solutions** related to their project topic.

Structure of Literature Review

A literature review usually includes the following components:

Overview of Existing Studies

This section describes **previous research related to the project topic**.

Example:
Researchers have developed different management systems to improve efficiency in data management and reduce manual errors.

Summary of Previous Work

The researcher summarizes **important findings from previous studies**.

Example:
Several studies have shown that computerized management systems significantly improve data accuracy and reduce processing time.

Analysis of Methods and Technologies

This section explains the **methods, technologies, and approaches used in previous research**.

Example:
Most previous systems use database management systems, web technologies, and user authentication to manage data securely.

Limitations of Previous Research

The literature review also identifies **weaknesses or limitations in existing systems**.

Example:
Some systems lack user-friendly interfaces, while others do not support advanced reporting features.

Research Gap

This part highlights **what is missing in previous studies** and why the current project is needed.

Example:
Although many systems exist, there is still a need for a more efficient and user-friendly system that integrates multiple functionalities in one platform.

Example Literature Review (For a Project)

Literature Review

Many researchers have developed management systems to improve the efficiency of data handling in organizations. These systems aim to replace traditional manual methods with computerized solutions that provide faster and more accurate results.

According to several studies, automated management systems help reduce human errors and improve productivity. Researchers have used technologies such as databases, web applications, and cloud-based platforms to design systems that store and manage large amounts of data efficiently.

In the case of library and student management systems, previous research has focused on developing systems that allow administrators to manage records, track activities, and generate reports. These systems have improved the organization of data and simplified administrative tasks.

However, some existing systems have limitations such as complex interfaces, limited functionality, and lack of integration with modern technologies. These limitations highlight the need for improved systems that are more flexible, user-friendly, and efficient.

Therefore, the current project aims to address these challenges by developing a system that provides better usability, improved data management, and enhanced system functionality.

Importance of Literature Review

A literature review is important because it:

- Provides **background knowledge** about the research topic
- Shows **understanding of existing work**
- Helps identify **research gaps**
- Supports the **need for the project**
- Improves the **quality and credibility of research**

6. Methodology

The methodology used in this project follows a **modular system design approach**, where the access control system is divided into several functional components. Each component performs a specific role in evaluating access requests and enforcing security policies.

The first step in the process is **user authentication**. The user enters their login credentials through a web interface. These credentials are verified against the stored records in the system database.

After successful authentication, the system collects **contextual information** related to the access request. This includes parameters such as device security status, user role, geographic location, and application risk level.

Next, the **Risk Engine** evaluates these contextual parameters and calculates a risk score. Each factor contributes a specific value to the overall risk score based on predefined rules.

Once the risk score is calculated, the **Policy Engine** applies security policies to determine the appropriate access decision. Depending on the risk level, the system may grant full access, restrict access to certain resources, or deny access completely.

Finally, the system displays the access decision to the user through the dashboard interface.

This modular approach ensures that the system is flexible and can be easily extended in the future to include additional security features such as multi-factor authentication or machine learning-based risk analysis.

Methodology refers to the **systematic approach or process used to complete a project or research study**. It explains the **methods, techniques, tools, and procedures** used to design, develop, and implement the project.

In simple terms, methodology answers the question:

- **“How will the project be carried out?”**

It describes the **step-by-step process followed during the project development**. A clear methodology helps ensure that the project is organized, structured, and completed efficiently.

2⃣ Purpose of Methodology

The methodology section is important because it:

✓ Explains the **process used to develop the project**
✓ Shows the **techniques and tools used in the project**
✓ Helps others **understand and reproduce the project process**
✓ Ensures the project follows a **structured development approach**
✓ Improves the **accuracy and reliability of the project results**

Project Development Methodology

Most software or system projects follow a **System Development Life Cycle (SDLC)** or a structured development process. The common stages are:

Requirement Analysis

This is the **first stage of the project** where the project requirements are identified and analyzed.

In this phase:

The problem is clearly defined.

User requirements are collected.

System needs and objectives are identified.

Example:
For a **Student Management System**, the requirements may include:

Student registration

Storing student details

Managing academic records

Generating reports

This stage helps the development team **understand what the system needs to do**.

System Design

In this phase, the **structure and architecture of the system** are planned.

It includes:

Designing the system workflow

Creating database structure

Designing user interfaces

Preparing system diagrams

The design stage provides a **blueprint for the system development**.

Common design tools include:

Flowcharts

Data Flow Diagrams (DFD)

Entity Relationship Diagrams (ERD)

UML diagrams

3. System Development (Implementation)

In this stage, the **actual coding and development of the system** takes place.

Developers create the system using programming languages and tools such as:

Python

Java

PHP

HTML/CSS

MySQL or other databases

The system features are implemented based on the **design and requirements** defined earlier.

4. System Testing

After development, the system is tested to ensure it works correctly.

Testing helps identify:

Errors

Bugs

System performance issues

Types of testing include:

Unit Testing

Integration Testing

System Testing

User Acceptance Testing

This phase ensures the system is **reliable and functions as expected**.

System Implementation

In this stage, the system is **installed and used by the users**.

Activities include:

Installing the system

Migrating data if necessary

Training users

Deploying the system

The system becomes **fully operational** during this phase.

Maintenance and Support

After implementation, the system may require **updates, improvements, or bug fixes**.

Maintenance ensures:

The system continues to work efficiently.

Problems are corrected.

New features can be added if needed.

Tools and Technologies Used

The methodology section may also describe the **technologies used in the project**, such as:

Programming Languages

Database Systems

Development Tools

Frameworks

Hardware and Software requirements

Example:

Programming Language: Python / Java
Database: MySQL
Front-end: HTML, CSS, JavaScript

Example Methodology (For Project Report)

Methodology

The project follows a structured development approach to ensure successful system implementation. The development process begins with requirement analysis, where the system requirements and objectives are identified. Information is gathered from users to understand the problems in the existing system.

After collecting the requirements, the system design phase is carried out. In this stage, the system architecture, database structure, and user interface are designed to provide an efficient and user-friendly system.

The next stage involves system development, where the system is implemented using appropriate programming languages and development tools. The system features are developed according to the requirements and design specifications.

Once development is completed, the system undergoes testing to identify and correct errors. Different testing methods are applied to ensure that the system functions properly and meets user requirements.

Finally, the system is implemented and deployed for use. Maintenance and support activities are performed to ensure the system continues to operate effectively and efficiently.

7. System Design

System Design is an important stage in the development of any project or information system. It refers to the process of defining the architecture, components, modules, interfaces, and data of a system in order to satisfy the specified requirements. After the requirements of the system are collected and analyzed, the next step is to design how the system will work and how different parts of the system will interact with each other. System design acts as a blueprint for the development phase because it provides a clear structure and plan that guides programmers and developers during implementation.

The main purpose of system design is to transform the system requirements into a detailed design that shows how the system will operate. It focuses on organizing the system components in such a way that the system becomes efficient, reliable, and easy to maintain. During this stage, developers determine how data will flow through the system, how the system will store and retrieve information, and how users will interact with the system through interfaces. A well-designed system ensures that the final product meets user needs and performs its functions smoothly.

System design usually includes the design of system architecture. System architecture describes the overall structure of the system and how different components such as hardware, software, and databases work together. It provides a high-level overview of how the system is organized and how different modules communicate with each other. This helps developers understand the relationships between various parts of the system and ensures that all components are integrated properly.

Another important part of system design is database design. In most information systems, data is stored and managed using a database. Database design involves defining the structure of the database, including tables, fields, relationships, and constraints. A properly designed database ensures that data can be stored efficiently and retrieved quickly when needed. It also helps maintain data integrity and reduces redundancy by organizing information in a logical and structured manner.

User interface design is also a significant aspect of system design. This part focuses on how users interact with the system. The interface should be simple, clear, and user-friendly so that users can easily understand how to perform tasks within the system. Good interface design improves user experience and reduces the chances of errors while using the system. It involves designing screens, forms, menus, and navigation structures that allow users to interact with the system efficiently.

System design also involves defining system processes and workflows. This includes identifying how data moves through the system and how different operations are performed. Tools such as flowcharts, data flow diagrams, and entity relationship

diagrams are often used to represent these processes visually. These diagrams help developers understand the logic and flow of the system before the actual coding begins.

Security considerations are also included in system design. The system must ensure that sensitive information is protected from unauthorized access. This may involve implementing authentication mechanisms such as usernames and passwords, access control systems, and data encryption methods. Designing security measures at this stage helps prevent potential risks and protects the integrity of the system.

Another aspect of system design is ensuring system performance and scalability. The design should allow the system to handle increasing amounts of data and users without affecting performance. Developers must consider factors such as response time, system load, and resource usage when designing the system. A well-designed system should be flexible enough to accommodate future updates and improvements.

In conclusion, system design is a critical stage in the system development process because it provides a detailed plan for building the system. It translates user requirements into a structured design that guides the development team during implementation. A well-prepared system design improves system efficiency, enhances user experience, and ensures that the system can operate reliably and securely. By carefully designing the system before development begins, organizations can reduce errors, save development time, and produce a high-quality final product.

The system design follows a **modular architecture**, where different modules are responsible for specific tasks. This design approach improves system flexibility and maintainability.

The main modules in the system include:

User Authentication Module

Handles login verification and ensures that only authorized users can proceed to the next stage.

Risk Assessment Module

Calculates the contextual risk score by analyzing multiple parameters.

Policy Enforcement Module

Applies predefined access control policies based on the calculated risk score.

Database Module

Stores user information and supports data retrieval during authentication.

Each module communicates with the others to complete the authentication and access control process.

8. Database Design

Database Design is a fundamental part of system development that focuses on organizing and structuring data in a way that allows it to be stored, managed, and retrieved efficiently. It involves planning how data will be stored in a database and how different types of data will relate to each other. A well-designed database ensures that information is accurate, consistent, secure, and easy to access when needed. Database design plays a crucial role in information systems because most applications depend on databases to store and manage large amounts of data.

The process of database design begins after the system requirements are clearly understood. At this stage, developers analyze the type of information the system needs to store and how users will interact with that information. Based on this analysis, they determine what data elements are required and how those elements should be structured. The goal is to create a logical structure that supports the functions of the system while maintaining efficiency and data integrity.

In database design, data is typically organized into tables. Each table represents a specific type of information, such as users, products, transactions, or records. Within each table, information is stored in rows and columns. Each row represents a single record, while each column represents a specific attribute or field of that record. For example, in a student management system, a student table may contain fields such as student ID, name, date of birth, course, and contact information.

Relationships between tables are an important aspect of database design. In many systems, data in one table is related to data in another table. Establishing relationships allows the system to connect related information without storing duplicate data. For instance, in a library management system, a table containing book details may be connected to another table that records borrowing transactions. These relationships help maintain consistency and make it easier to retrieve related information when needed.

Another important concept in database design is normalization. Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. By dividing large tables into smaller, related tables and defining clear relationships between them, normalization helps ensure that each piece of data is stored only once. This reduces the chances of errors and makes the database more efficient and easier to maintain.

Database design also involves defining keys that uniquely identify records in a table. A primary key is used to uniquely identify each record within a table. For example, a student ID can serve as the primary key in a student table. Foreign keys are used to create relationships between tables by referencing the primary key of another table. These keys

help maintain the connections between different sets of data and ensure that relationships remain consistent.

Security and access control are also considered during database design. Since databases often store sensitive information, it is important to control who can access or modify the data. Database systems may include authentication mechanisms and permission settings that restrict access to authorized users. Proper security measures protect the data from unauthorized access, loss, or corruption.

Another factor considered in database design is performance. The database must be designed in such a way that it can handle large amounts of data and allow fast retrieval of information. Efficient indexing techniques and optimized query structures help improve the speed and performance of the database system. A well-designed database ensures that users can quickly access the information they need without delays.

In addition to the logical design of the database, developers also consider the physical design. Physical design refers to how the data will actually be stored on storage devices. It includes decisions about storage structures, indexing methods, and database management systems. These decisions influence the efficiency, reliability, and scalability of the database.

In conclusion, database design is a critical process that determines how data is structured, stored, and managed within a system. A carefully designed database improves data organization, ensures accuracy and consistency, and enhances the overall performance of the system. By planning the database structure properly, developers can create systems that efficiently handle large volumes of data while maintaining reliability and security.

The system uses **SQLite database** to store user credentials and roles.

Users Table

Field Description

Field	Description
id	Unique user identifier
username	User login name
password	User password
role	User role (admin, employee, contractor)

Database Operations

The system performs the following database operations:

User Verification

The login credentials entered by the user are compared with stored records.

User Role Retrieval

After successful authentication, the user's role is retrieved to determine risk level.

SQLite is chosen for this project because it is lightweight, easy to use, and suitable for small-scale applications.

9. Implementation

The system is implemented using the **Python Flask framework**, which allows rapid development of web applications.

The project is divided into several Python modules.

app.py

This file contains the main Flask application. It handles user login requests, processes form data, and coordinates communication between different modules.

risk_engine.py

This module calculates the risk score based on contextual parameters such as device status, location, application risk, and user role.

policy_engine.py

This module evaluates the calculated risk score and determines the appropriate access decision.

database.py

This module initializes the SQLite database and manages user records.

HTML Templates

The user interface is implemented using HTML templates stored in the templates folder.

Implementation is a crucial phase in the system development process where the designed system is converted into a working system. During this stage, the plans, designs, and specifications developed in earlier phases are put into action. Implementation involves developing the actual system, installing it in the working environment, and

making it ready for users. It ensures that the system operates according to the defined requirements and performs the tasks it was designed to accomplish.

The implementation phase begins after the system design is completed. In this stage, developers start translating the system design into code using appropriate programming languages, tools, and technologies. The different modules and components of the system are developed and integrated to create a fully functional system. Each module performs specific tasks and works together with other modules to ensure that the system functions smoothly.

One of the main activities during implementation is coding. Coding involves writing the instructions that allow the system to perform its functions. Developers follow the system design documents to ensure that the developed system meets the required specifications. Proper coding practices are followed to maintain code quality, readability, and maintainability. This helps ensure that the system can be easily updated or modified in the future if necessary.

Another important aspect of implementation is system integration. In most systems, multiple modules or components are developed separately. During implementation, these modules are combined and integrated to work together as a complete system. Integration ensures that different parts of the system communicate properly and share data without errors. This step is essential for ensuring that the system operates as a unified solution rather than as separate components.

Testing also plays an important role during implementation. As the system is developed, it is tested to identify and correct errors or bugs. Testing ensures that the system performs its functions correctly and meets user requirements. Different types of testing may be carried out, such as unit testing, integration testing, and system testing. These tests help ensure that the system is reliable, stable, and free from major errors before it is delivered to users.

Once the system has been developed and tested successfully, it is installed in the actual working environment. This process is known as system deployment or system installation.

During this stage, the system is set up on the required hardware and software platforms so that users can begin using it. Necessary configurations are completed, and the system is prepared for operational use.

User training is another important activity during the implementation phase. Since users may not be familiar with the new system, they need guidance on how to operate it effectively. Training sessions or user manuals may be provided to help users understand the features and functions of the system. Proper training helps users adapt to the new system and ensures that they can use it efficiently.

Data conversion or data migration may also occur during implementation. If the organization previously used a manual system or an older computerized system, existing data may need to be transferred to the new system. This process ensures that important historical information is not lost and can continue to be used within the new system.

After the system is implemented and users begin working with it, monitoring and support activities are carried out to ensure that the system functions properly. Any issues or unexpected problems that arise are addressed quickly. Continuous monitoring helps maintain system performance and ensures that the system meets user expectations.

In conclusion, implementation is the stage where the planned system becomes a real, operational system. It involves coding, integrating system components, testing, installing the system, and training users. A successful implementation ensures that the system functions correctly, satisfies user needs, and provides the intended benefits to the organization. Proper planning and execution during this phase are essential for achieving a reliable and efficient system.

10. Testing and Results

Testing and Results is an important phase in the development of any system or project. It is the stage where the developed system is carefully examined to ensure that it functions correctly and meets the specified requirements. Testing helps identify errors, bugs, and problems that may exist in the system before it is used by real users. The main goal of testing is to ensure that the system is reliable, efficient, and capable of performing all the intended tasks without failure.

Testing is carried out after the system has been developed and integrated. During this stage, the system is executed with different types of input data to observe how it behaves and to verify whether it produces the expected output. This process helps developers confirm that each component of the system works properly and that the system as a whole performs its functions accurately. If any problems are found during testing, they are corrected and the system is tested again until it performs satisfactorily.

One of the first forms of testing conducted is unit testing. In this stage, individual components or modules of the system are tested separately to ensure that each part functions correctly. By testing modules independently, developers can easily identify the specific location of errors and fix them effectively. Once the individual modules pass the testing process, they are combined together for further evaluation.

After unit testing, integration testing is performed. Integration testing focuses on verifying that different modules of the system work together properly. When modules are integrated, they need to communicate and share data with each other. Integration testing ensures that this interaction occurs without errors and that the system operates smoothly as a complete unit.

System testing is another important step in the testing process. In system testing, the entire system is tested as a whole to ensure that it meets the overall requirements and performs all the necessary functions. This type of testing examines the system's performance, functionality, and reliability. It checks whether the system responds correctly to user inputs and produces accurate results.

User acceptance testing may also be carried out before the system is officially implemented. In this stage, the system is tested by actual users or stakeholders to determine whether it satisfies their needs and expectations. Users interact with the system in a real or simulated working environment and provide feedback regarding its usability and functionality. This feedback helps ensure that the system is practical and suitable for real-world use.

During the testing process, different test cases are prepared to evaluate the system's performance under various conditions. These test cases include specific input data, expected results, and the actual results produced by the system. By comparing the expected results with the actual output, developers can determine whether the system functions correctly. If differences are observed, adjustments and corrections are made to improve the system.

The results obtained from testing demonstrate the effectiveness and reliability of the system. When the system passes all testing stages successfully, it indicates that the system is functioning as intended. The results confirm that the system can handle user operations, process data accurately, and provide correct outputs. Positive testing results also indicate that the system is stable and ready for implementation.

Testing results may also highlight improvements made by the new system compared to previous methods. For example, the system may process information faster, reduce manual errors, and provide more organized data management. These improvements demonstrate the benefits and effectiveness of the developed system.

In conclusion, testing and results are essential components of the system development process. Testing ensures that the system works correctly and meets all functional requirements, while the results confirm the system's reliability and performance. By carefully evaluating the system through different testing stages, developers can deliver a high-quality system that satisfies user needs and operates efficiently in a real-world environment.

Testing was performed to evaluate how the system responds to different contextual scenarios.

Test Case 1 – Secure Environment

Device Status: Patched
Location: Office
Application Risk: Low

Risk Score: 0

Result: **Full Access Granted**

Test Case 2 – Moderate Risk

Device Status: Unpatched
Location: Public Network
Application Risk: Medium

Risk Score: 45

Result: **Restricted Access**

Test Case 3 – High Risk

Device Status: Unpatched
Location: Foreign Location
Application Risk: High

Risk Score: 95

Result: **Access Denied**

11. Conclusion

The **Context-Aware Zero Trust Access Control System** developed in this project demonstrates how modern security principles can be applied to improve authentication and access control mechanisms.

Unlike traditional authentication systems that rely solely on passwords, this system evaluates contextual parameters such as device status, location, application risk level, and user role. By calculating a risk score and enforcing access control policies, the system can dynamically determine whether a user should be granted access.

The implementation highlights the advantages of **Zero Trust security architecture**, including improved protection against unauthorized access and enhanced security awareness.

Although the system is a prototype, it provides a strong foundation for developing more advanced Zero Trust solutions in the future.

12. IEEE References

[1] J. Kindervag, “Build Security Into Your Network's DNA: The Zero Trust Network Architecture,” Forrester Research, 2010.

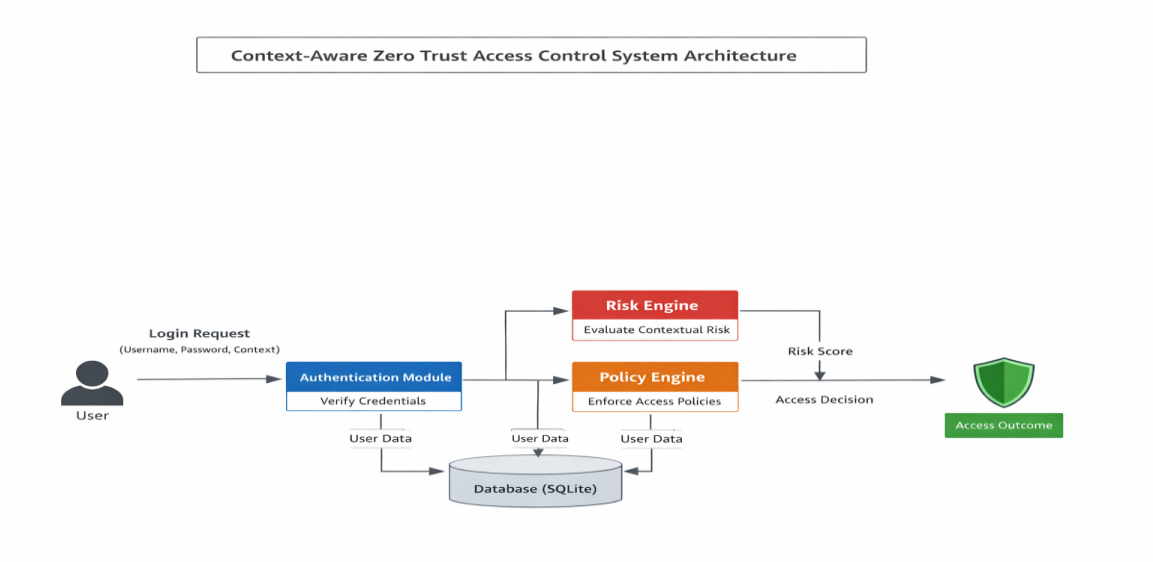
[2] National Institute of Standards and Technology, “Zero Trust Architecture,” NIST Special Publication 800-207, 2020.

[3] W. Stallings, *Network Security Essentials: Applications and Standards*, 6th Edition, Pearson, 2017.

[4] E. Cole, *Advanced Persistent Threat: Understanding the Danger and How to Protect Your Organization*, Syngress, 2012.

[5] A. Singh and J. Singh, “Zero Trust Security Model: A New Approach to Network Security,” *International Journal of Computer Applications*, vol. 179, no. 45, pp. 20-24, 2018.

Architecture Diagram



System Architecture

The **Context-Aware Zero Trust Access Control System** follows a modular architecture designed to evaluate user access requests based on contextual information and predefined security policies. The system architecture consists of several interconnected components that work together to ensure secure access control.

The main components of the system include:

1. User Interface

The user interface allows users to interact with the system. It provides a login page where users enter their credentials and contextual parameters such as device status, location, and application risk level.

2. Authentication Module

This module verifies the user credentials by comparing the entered username and password with the records stored in the database.

3. Risk Engine

The Risk Engine evaluates contextual information associated with the access request. It

assigns risk values based on factors such as device security status, location, and application risk.

4. Policy Engine

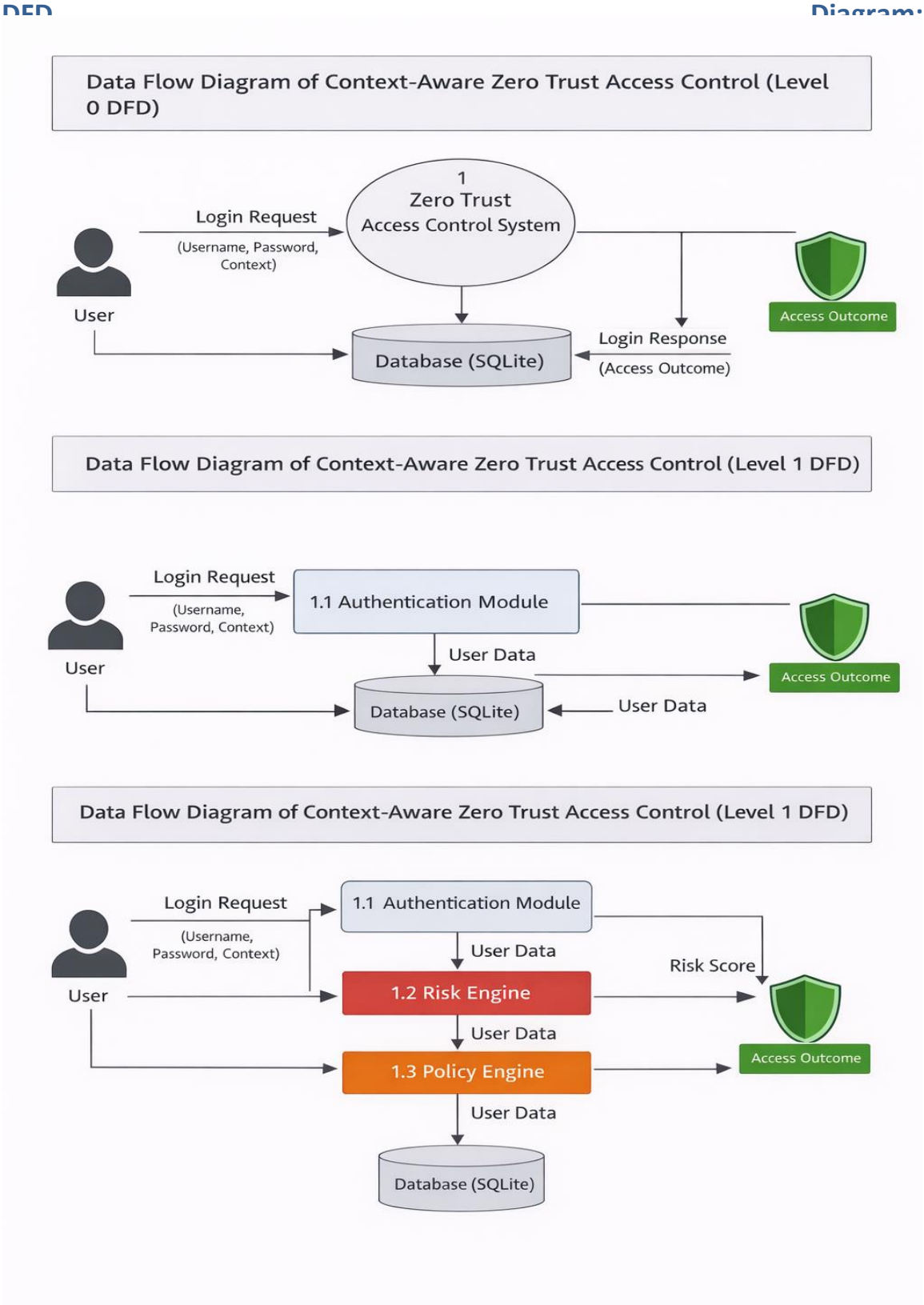
The Policy Engine uses the risk score calculated by the Risk Engine to determine the appropriate access level.

5. Database

The database stores user credentials, roles, and other relevant information required for authentication.

The interaction between these components ensures that every access request is verified and evaluated before granting system access.

DFD Diagram:


Data Flow Diagram of Context-Aware Zero Trust Access Control (Level 0 DFD)
1
Zero Trust
Access Control System
Login Request
(Username, Password, Context)
User
Access Outcome
Database (SQLite)
Login Response
(Access Outcome)
Data Flow Diagram of Context-Aware Zero Trust Access Control (Level 1 DFD)
Login Request
(Username, Password, Context)
1.1 Authentication Module
User
User Data
Access Outcome
Database (SQLite)
User Data
Data Flow Diagram of Context-Aware Zero Trust Access Control (Level 1 DFD)
Login Request
(Username, Password, Context)
1.1 Authentication Module
User
User Data
Risk Score
1.2 Risk Engine
User Data
1.3 Policy Engine
Access Outcome
User Data
Database (SQLite)

DFD explanation:

A **Data Flow Diagram (DFD)** illustrates how data moves within the system.

Level 0 DFD (Context Diagram)

In the Level 0 DFD, the entire system is represented as a single process called the **Zero Trust Access Control System**.

The user sends a **login request** containing the username, password, and contextual parameters. The system processes this request and communicates with the database to verify user information. After evaluating the risk level, the system returns an **access decision** to the user.

Level 1 DFD

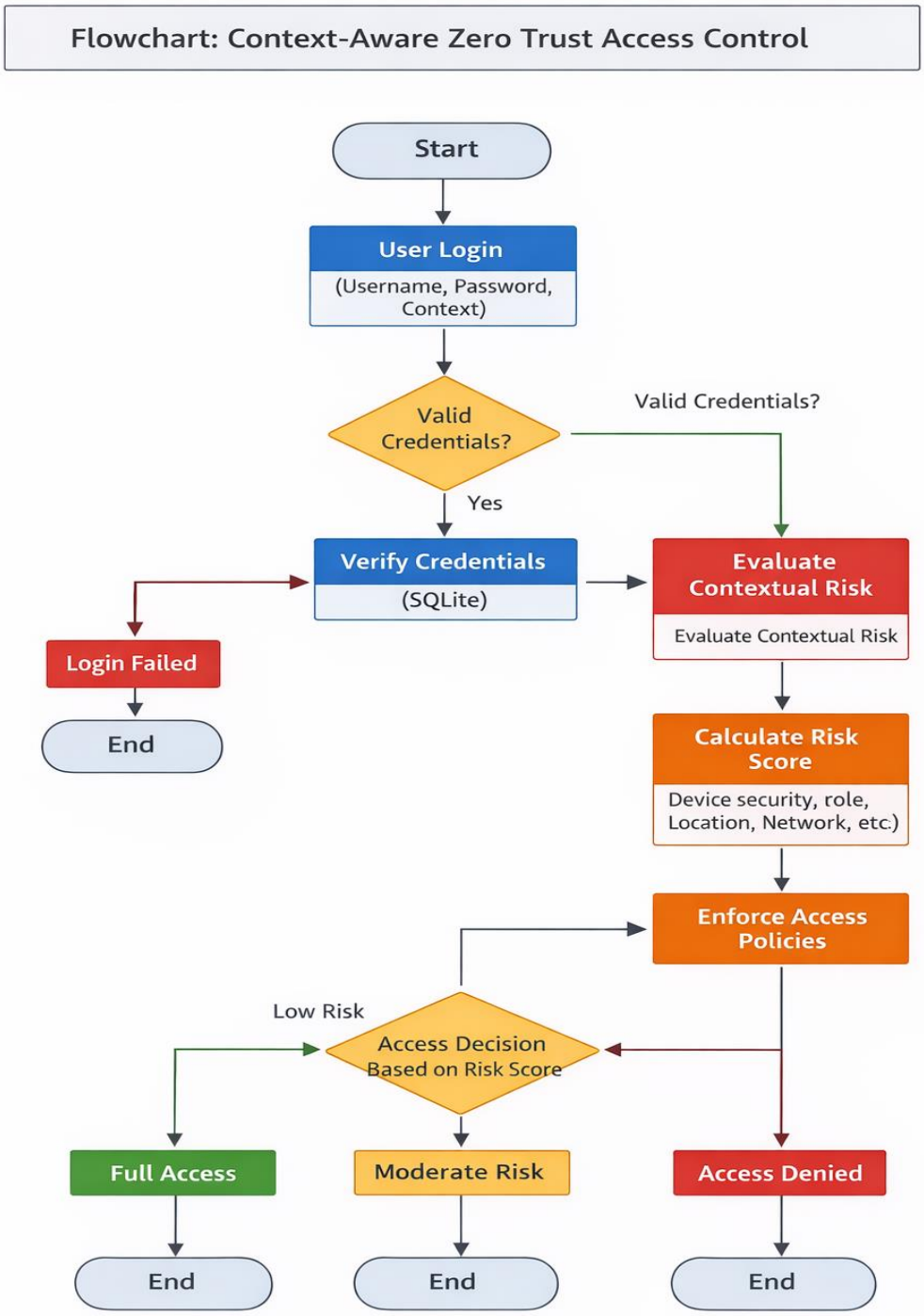
The Level 1 DFD breaks the system into smaller processes:

1. Authentication Module
Receives login credentials and verifies them with the database.

2. Risk Engine
Analyzes contextual parameters and calculates a risk score.

3. Policy Engine
Uses the calculated risk score to determine whether access should be granted or denied.

The data flows between these components and the database to complete the authentication and access control process


Flowchart: Context-Aware Zero Trust Access Control
Start
User Login
(Username, Password, Context)
Valid Credentials?
Valid Credentials?
Yes
Verify Credentials
(SQLite)
Evaluate Contextual Risk
Evaluate Contextual Risk
Login Failed
End
Calculate Risk Score
Device security, role, Location, Network, etc.)
Enforce Access Policies
Low Risk
Access Decision Based on Risk Score
Full Access
Moderate Risk
Access Denied
End
End
End

Flow chart explanation:

The flowchart describes the step-by-step process of how the system handles user login and access decisions.

The process begins when a user enters their login credentials.

The system verifies the credentials using the database.

If the credentials are invalid, the system denies access and terminates the process.

If the credentials are valid, the system proceeds to evaluate contextual risk.

The Risk Engine calculates a risk score based on contextual parameters.

The Policy Engine analyzes the risk score.

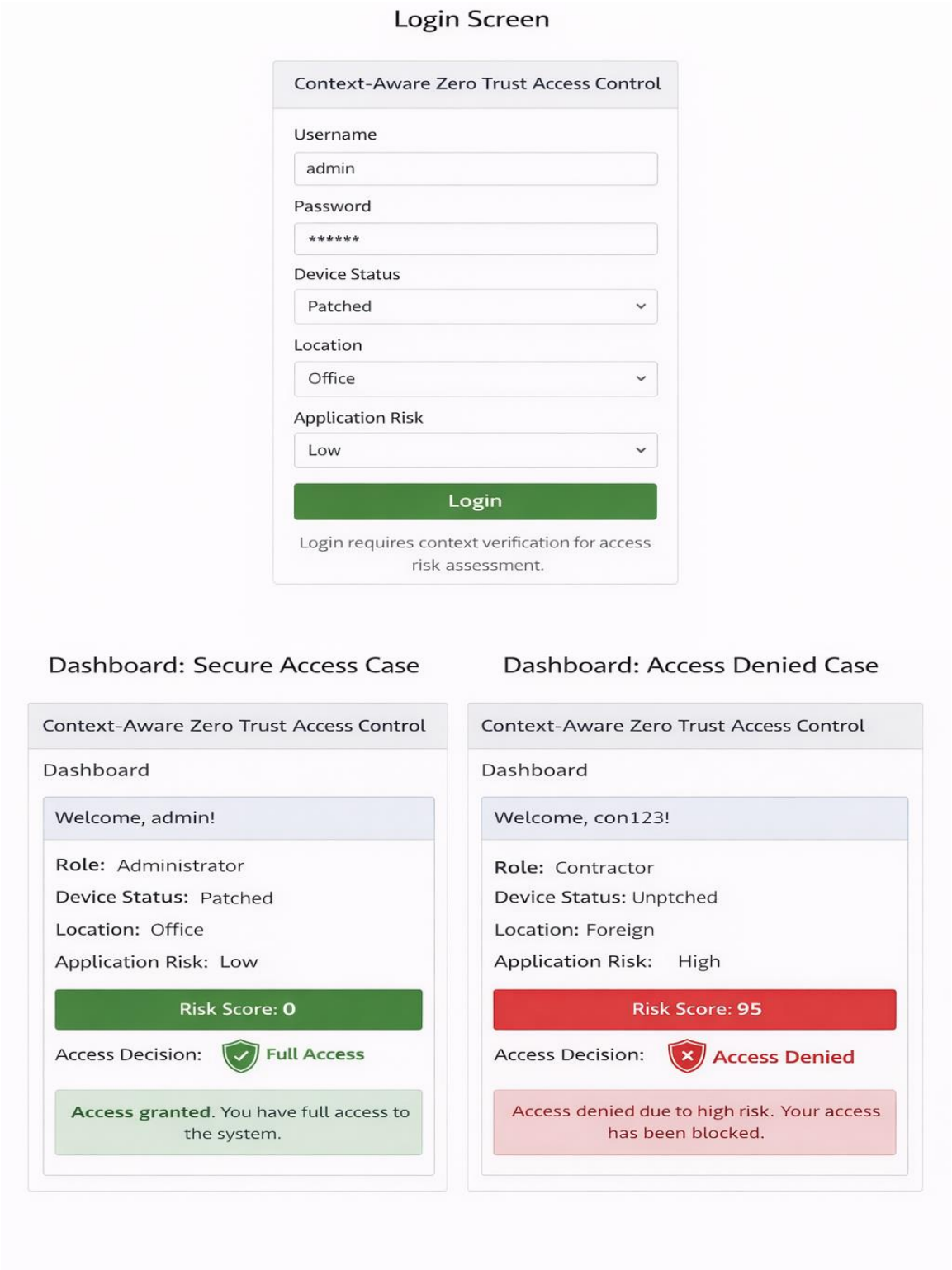
If the risk score is low, full access is granted.

If the risk score is moderate, restricted access may be applied.

If the risk score is high, access is denied.

This flow ensures that security decisions are made dynamically based on contextual risk.

User Interface Screenshots



User Interface Explanation

The system provides a simple and user-friendly web interface built using **HTML and CSS** within the Flask framework.

Login Page

The login page allows users to enter the following information:

Username

Password

Device status (patched or unpatched)

Location (office, remote, foreign)

Application risk level (low, medium, high)

Once the user submits this information, the system processes the request.

Dashboard Page

After authentication, the dashboard displays the following information:

Username

User role

Device status

Location

Application risk level

Calculated risk score

Access decision

If the risk score is low, the system displays **Full Access Granted**. If the risk score is high, the system displays **Access Denied**.

The interface helps users clearly understand the security decision made by the system.

Coding:

```
from flask import Flask, render_template, request

import sqlite3

from risk_engine import calculate_risk

from policy_engine import access_decision


app = Flask(__name__)


def check_user(username,password):


    conn = sqlite3.connect("database.db")

    cursor = conn.cursor()


    cursor.execute("SELECT * FROM users WHERE username=? AND password=?",
(username,password))

    user = cursor.fetchone()


    conn.close()


    return user


@app.route("/", methods=["GET","POST"])

def login():


    if request.method == "POST":
```

```
username = request.form["username"]
password = request.form["password"]

device = request.form["device"]
location = request.form["location"]
app_risk = request.form["app_risk"]

user = check_user(username,password)

if user:

    role = user[3]

    score = calculate_risk(role,device,location,app_risk)

    decision = access_decision(score)

    return render_template("dashboard.html",
                           username=username,
                           role=role,
                           risk_score=score,
                           decision=decision)

else:
    return "Invalid Login"
```

```
    return render_template("login.html")



if __name__ == "__main__":
    app.run(debug=True)
```

Coding explanation:

Importing Required Libraries

The code begins by importing the necessary modules.

```
from flask import Flask, render_template, request
```

Flask is a lightweight Python web framework used to build web applications.

`Flask` creates the application instance.

`render_template` is used to display HTML pages stored in the templates folder.

`request` is used to collect data sent by the user through web forms.

```
import sqlite3
```

This module is used to connect and interact with an **SQLite database**. SQLite is a lightweight database that stores data in a single file.

```
from risk_engine import calculate_risk
```

This imports the **calculate_risk function** from another Python file called `risk_engine.py`. This function calculates a **risk score** based on parameters such as role, device, location, and application risk.

```
from policy_engine import access_decision
```

This imports the **access_decision function** from `policy_engine.py`. This function uses the calculated risk score to determine whether the user should be **allowed or denied access**.

Creating the Flask Application

```
app = Flask(__name__)
```

This line creates the main Flask application object.
`__name__` tells Flask where the application is located so it can load resources like templates and static files correctly.

Function for Checking User Credentials

The function `check_user(username, password)` verifies if a user exists in the database.

Inside the function, a connection to the SQLite database named **database.db** is created using:

```
conn = sqlite3.connect("database.db")
```

A cursor object is created using:

```
cursor = conn.cursor()
```

The cursor allows the program to execute SQL queries on the database.

Next, the SQL query is executed:

```
cursor.execute("SELECT * FROM users WHERE username=? AND password=?",
(username,password))
```

This query searches the **users table** for a record where the username and password match the values provided by the user.
The `?` placeholders are used to safely insert user input into the query, which helps prevent SQL injection attacks.

```
user = cursor.fetchone()
```

This retrieves the first matching row from the query result. If a user exists, it returns the user record; otherwise it returns `None`.

After the query is completed, the database connection is closed using:

```
conn.close()
```

Finally, the function returns the user record.

Creating the Login Route

```
@app.route("/", methods=["GET","POST"])
```

This decorator defines the route for the main page of the application.
The route accepts both **GET** and **POST** requests.

GET request means the user is simply opening the login page.
POST request means the user has submitted the login form.

Login Function

```
def login():
```

This function handles the login logic.

First, the program checks whether the request method is POST.

```
if request.method == "POST":
```

This means the user has submitted the login form.

Getting User Input from the Form

The following lines retrieve the values submitted by the user through the HTML form.

```
username = request.form["username"]
password = request.form["password"]
```

These collect the login credentials entered by the user.

Additional contextual information is also collected.

```
device = request.form["device"]
location = request.form["location"]
app_risk = request.form["app_risk"]
```

These values represent the **context of the login attempt**, which will be used for risk evaluation.

For example:

Device type (mobile, laptop, unknown device)

Location of login (trusted location or unknown location)

Application risk level

Checking the User in the Database

```
user = check_user(username,password)
```

This calls the previously defined function to verify if the username and password exist in the database

If User Exists

```
if user:
```

If the database returns a valid user record, the program continues.

```
role = user[3]
```

This extracts the user's role from the database record.

For example:

Admin

Employee

Manager

The role is stored in the **fourth column** of the users table.

Calculating the Risk Score

```
score = calculate_risk(role,device,location,app_risk)
```

This function calculates a **risk score** using several parameters:

User role
Device type
Login location
Application risk level

The risk engine evaluates these factors and returns a numeric score representing how risky the login attempt is.

Making an Access Decision

```
decision = access_decision(score)
```

The policy engine uses the risk score to decide the access result.

Typical decisions might include:

Allow access

Require additional verification

Deny access

Displaying the Dashboard

```
return render_template("dashboard.html", username=username, role=role, risk_score=score, decision=decision)
```

If login and risk evaluation are successful, the system loads the **dashboard.html** page.

The following information is passed to the dashboard:

Username
User role
Calculated risk score
Final access decision

The HTML page can display these values dynamically.

If Login Fails

```
else: return "Invalid Login"
```

If the username and password do not match any record in the database, the system displays an **Invalid Login** message.

Showing the Login Page

```
return render_template("login.html")
```

If the request is a GET request, the application simply loads the **login.html** page where the user can enter login details.

Running the Application

```
if __name__ == "__main__":
```

This ensures the application runs only when the script is executed directly.

```
app.run(debug=True)
```

This starts the Flask development server.

The `debug=True` option enables debugging mode, which automatically reloads the server when code changes and shows detailed error messages.

Overall Workflow of the Program

The user opens the login page in the browser.

The user enters username, password, device type, location, and application risk.

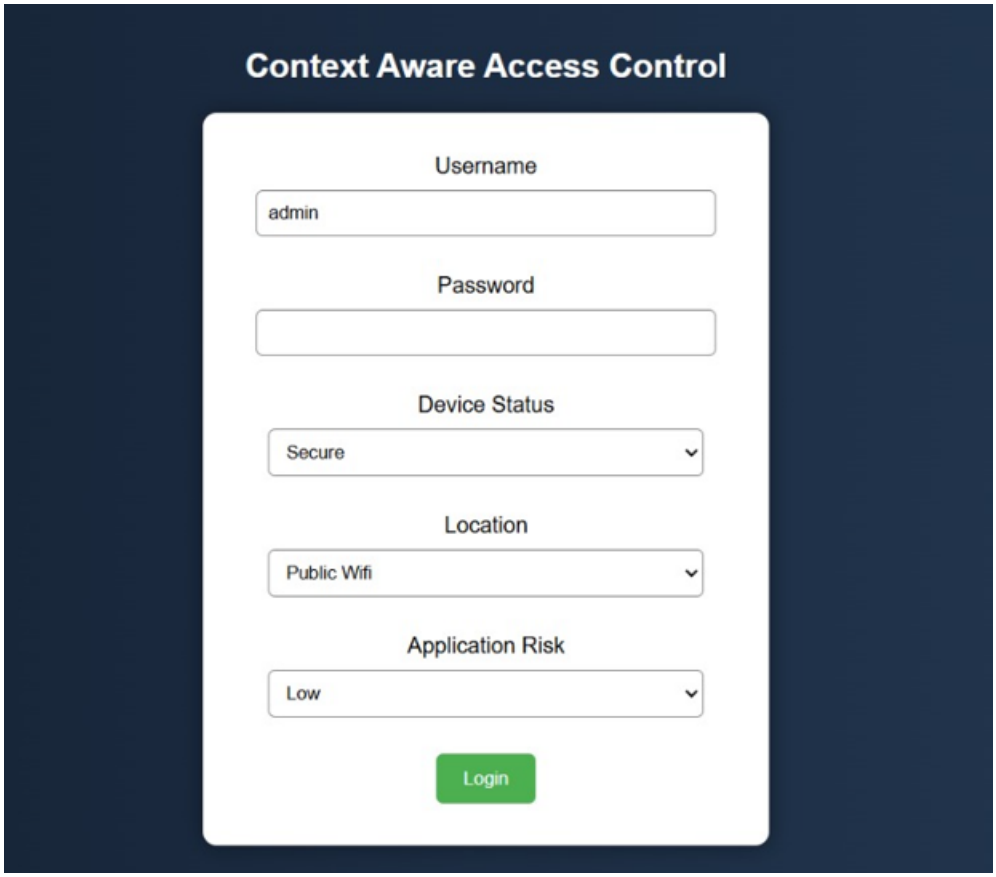
The system verifies the credentials using the SQLite database.

If the user exists, the system calculates a risk score using contextual information.

A policy engine evaluates the risk score and decides whether access should be allowed.

The result is displayed on the dashboard page.

INPUT:



The image shows a **login page for a Context-Aware Access Control System**. It collects user credentials and contextual information before granting access.

1. Username: Identifies the user account.

2. Password: Verifies the user's identity.

3. Device Status: Indicates whether the device is *Secure* or *Unsecured*.

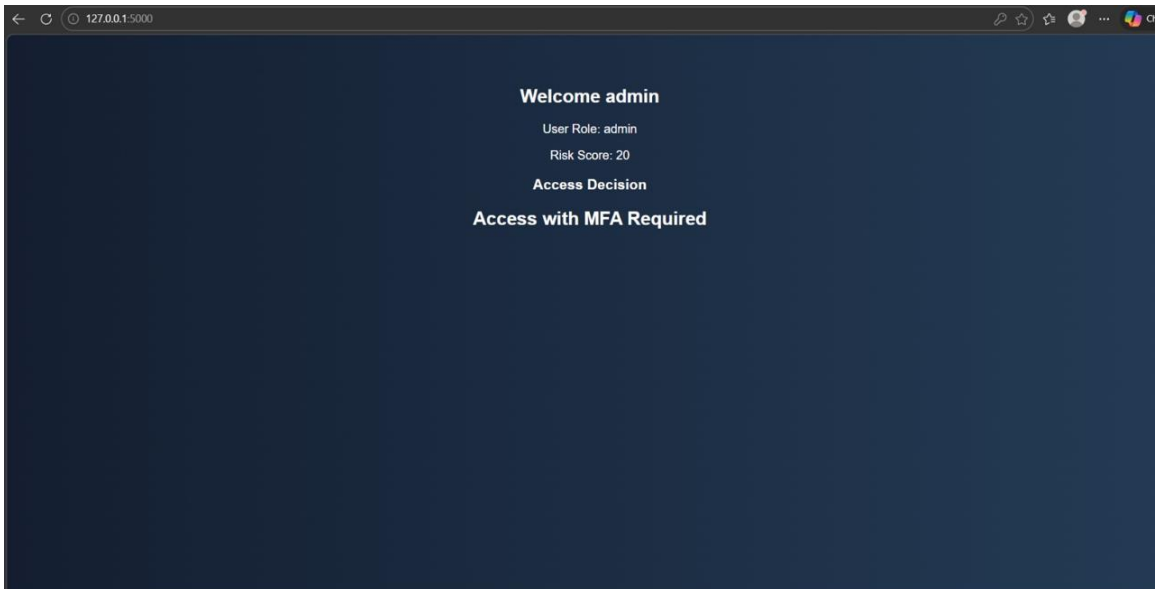
4. Location: Shows the network used (e.g., *Public WiFi* or *Office Network*).

5. Application Risk: Specifies the sensitivity level of the application (*Low, Medium, High*).

6. Login Button: Submits the information for verification.

The system uses both **credentials and context (device, location, risk)** to decide whether access should be allowed. □

OUTPUT:



This page displays the **final access decision** made by the **Context-Aware Access Control System** after a user logs in. It shows important details about the user, their risk level, and the security action required.

At the top, the message **"Welcome admin"** confirms that the user has successfully authenticated in the system. This means the entered **username and password were correct**, and the system recognized the user account.

Below that, **User Role: admin** indicates the permission level assigned to the user. The *admin role* usually has higher privileges, such as managing users, accessing sensitive data, or controlling system settings. Role information helps the system decide what resources the user is allowed to access.

The page also displays **Risk Score: 20**. This score is calculated by the system based on contextual factors provided during login, such as **device status, location, and application risk level**. Each factor contributes to the overall risk calculation. A lower score usually means safer access conditions, while a higher score indicates a more risky login attempt.

Next, the **Access Decision** section shows the system's final security decision. In this case, it states **"Access with MFA Required."** This means the user is allowed to continue, but must complete **Multi-Factor Authentication (MFA)** for additional verification. MFA adds another security layer by requiring something beyond the password, such as a **one-time password (OTP), authentication app code, or mobile verification**.

This decision typically occurs when the risk score is **moderate**. Instead of completely blocking access, the system increases security by asking for an additional verification step. This approach improves protection against unauthorized access while still allowing legitimate users to proceed.

Overall, this page demonstrates how the system combines **user role information and calculated risk scores** to make **dynamic and intelligent access control decisions**, ensuring both **system security and controlled user access**.